

## Electronic Companion

Accepted Service Adaptation:  
Compact Allocation Quotients and Minimal Price Memory

Anonymous manuscript for Operations Research

Revision R30, September 23, 2026

### EC.1 The Laminar Polymatroid Connection

This section supplies the feasible-set link to Federgruen and Groenevelt (1986); Groenevelt (1991); it is not offered as a new polymatroid theorem. In the variables of Proposition 2.1, set  $\ell_h = w_h a_{v(h)} l_{v(h)}$ ,  $u_h = z_h - \ell_h$ , and  $c_h = w_h a_{v(h)} (h_{v(h)} - l_{v(h)})$ . The residual subtree capacities are  $C_h = w_h B_{v(h)} - \sum_{j \succeq h} \ell_j \geq 0$ , and the residual total is  $b' = b_0 - \sum_h \ell_h \geq 0$ . Write  $\mathcal{T}_h$  for the descendant set at  $h$ . For  $A \subseteq \mathcal{T}_h$ , define the rank recursively by

$$f_h(A) = \min \left\{ C_h, c_h \mathbf{1}_{\{h \in A\}} + \sum_{j \text{ child of } h} f_j(A \cap \mathcal{T}_j) \right\}. \quad (1)$$

The root function  $f_0$  is normalized, nondecreasing, and submodular. Indeed, the expression inside the minimum is a sum of a nonnegative modular function and submodular functions on disjoint ground sets; truncation at a nonnegative constant preserves nondecreasing submodularity. The latter fact follows directly from diminishing marginal increments: the increment of a truncated function is its original nonnegative increment capped by the remaining distance to the truncation level, both of which cannot increase when the conditioning set grows.

The residual packing region is exactly  $P(f_0) = \{u \geq 0 : \sum_{h \in A} u_h \leq f_0(A) \text{ for every } A\}$ . To verify the rank, maximize allocation to a subset  $A$  on a subtree. The independent child capacities contribute their ranks and the current coordinate contributes  $c_h \mathbf{1}_{\{h \in A\}}$ ; the current cap truncates their sum. Any value between zero and that sum can be obtained by scaling a feasible nonnegative allocation, which proves (1). Conversely, the subset inequalities for singleton coordinates and for descendant sets imply the original box and subtree rows, since their ranks do not exceed the respective capacities. This proves equality of the regions.

Feasibility gives  $b' \leq f_0(\mathcal{T})$ . Define  $f^{b'}(A) = \min\{b', f_0(A)\}$ . The exact-payment feasible set is the base

$$\left\{ u \in P(f^{b'}) : \sum_h u_h = f^{b'}(\mathcal{T}) = b' \right\}.$$

Truncation matters when the promised total is below the maximum feasible allocation. The objective after translation remains separable strictly concave. Hence both the laminar and the polymatroid connections are exact, including nonzero local lower bounds and a root equality. Our input-compression and execution-memory claims concern this repeated-subtree subclass, not separable allocation over all polymatroids.

## EC.2 Reading Guide and Preservation of the General Theory

The main manuscript proves the laminar equivalence, the occurrence quotient, rational preprocessing bounds, minimal execution alphabet, heterogeneous frontier, dispersion comparison, and fixed-table supporting inequality. This companion supplies implementation details, additional boundary arguments, and reproducibility specifications. It is not a substitute for the central proofs.

The preceding manuscript and electronic companion are retained verbatim under the R30 predecessor directory and are supplied as separate retained-theory PDFs. All earlier revision directories remain unchanged. The preservation index maps each inherited result to its original location. This organization keeps the current paper centered on its quotient and memory conclusions without deleting the broader accepted-adaptation theory. In particular, the finite-price assumptions do not retroactively restrict the older convex, switching, promise-state, or certificate theorems.

The retained main manuscript contains accepted balance conditions, continuation-price transfers, release frontiers, comparator-adjusted continuation rents, switching-friction paths, information hierarchies, promise-state recursion, continuous-state envelopes, extensive-form-dual shared-table representability, reward-only and correlated cache certificates, and cache governance. Its companion retains stock-identification refinements, detailed proofs, active-set and dual checks, continuous-state arguments, and adaptive deployment bounds. Historical and computational supplements remain separate readable volumes. These materials support reuse of the general institution while the new theorems state their own narrower computational hypotheses explicitly.

## EC.3 Coefficient-Event Compilation

For each response, store ordered rational knots  $k_1 < \dots < k_s$  and one affine coefficient pair  $(m_i, c_i)$  on each interval, including the two tails. Equal adjacent pairs are removed. Evaluation uses binary search and the convention that a knot belongs to its right interval; continuity makes the value independent of that convention.

At vertex  $v$ , initialize the left-tail coefficient with  $a_v h_v + \beta \sum_w p_{vw} U_w$ . The local clipped payment changes slope and intercept at

$$(r_v - q_v h_v)/a_v, \quad (r_v - q_v l_v)/a_v.$$

For each successor knot, add the weighted difference between its new and old coefficient pair to the event at that knot. Combining equal-key events before sorting ensures that a repeated downstream breakpoint is stored once at the current vertex. Starting from the left-tail pair, sweep events to recover the pre-cap lines. This operation merges coefficients; it never recovers a slope from two evaluations at newly generated cap crossings.

If the cap is redundant, store the pre-cap curve. Otherwise find the first crossing of  $B_v$  by scanning the nonconstant affine pieces. Replace the entire prefix by the constant  $B_v$ , retain the original crossing line to its right, and discard any newly equal neighboring pairs. The crossing may coincide with an existing knot or lie

at the left endpoint of a constant equality plateau. Both cases use the same continuity convention.

The input checker requires rational integers or strings, positive payment and quadratic coefficients, a valid physical interval, positive outgoing probabilities summing to one, topological vertex order, and reachability from the root. Duplicate edges must first be aggregated. It rejects an empty feasible continuation interval or an out-of-range root promise rather than returning an approximate feasible answer. Floating-point primitives are rejected by the exact compiler because their intended rational interpretation would otherwise be ambiguous.

### EC.3.1 A direct plateau argument

Let  $x_1, x_2$  maximize  $G(x) - \eta_1 Y(x)$  and  $G(x) - \eta_2 Y(x)$  over the same compact convex pre-cap set, with  $\eta_1 < \eta_2$ . Optimality gives

$$G(x_1) - G(x_2) \geq \eta_1(Y(x_1) - Y(x_2)), \quad G(x_1) - G(x_2) \leq \eta_2(Y(x_1) - Y(x_2)).$$

If the two payments coincide, these inequalities imply equal rewards. Both actions then maximize both objectives. Strict concavity forces  $x_1 = x_2$ . A payment plateau therefore changes neither the optimal local tier nor any future tier, although the price supporting that plan may vary. This proves policy uniqueness at a cap-equality plateau without assuming uniqueness of a Lagrange multiplier.

At a response's upper tail, the implementation returns one finite price strictly below its first knot. At the lower tail, it returns an endpoint of the last decreasing segment; a completely constant response uses price zero. None of these choices changes the optimum. If the root itself is capped exactly at its promised payment, its cap multiplier and equality price may be redistributed, but their contribution to the objective remains the same because the two right-hand sides agree.

### EC.3.2 Explicit pseudocode

The following procedure is equivalent to the executable coefficient-event implementation.

Process vertices in reverse topological order. At each vertex, validate the feasible lower payment. Create the two local clipping events when the interval is nondegenerate. Add discounted probability-weighted successor coefficient changes into a common event dictionary. Sort its keys and sweep to obtain the pre-cap curve. When necessary, invert that curve at the current cap and replace its left prefix by a constant. Store the remaining lines and current barrier. After the root query arrives, invert the root curve at the exact promised payment. Propagate the maximum of the incoming price and current barrier, aggregate identical reached vertex-price pairs, and record actions and multipliers. Finally minimize the reached execution graph by backward behavioral signatures.

The procedure constructs all graph response curves, but it constructs execution states only for the supplied query. A different query may change both the reached pairs and the minimized machine. The table-storage bound and writable-alphabet bound describe different data structures.

## EC.4 Bit Bounds for Execution and Certificates

The main bit theorem deliberately concerns response preprocessing and one inversion. The next observation explains why exact execution remains polynomial while a printed total value can be much longer than an individual response coefficient.

**Proposition EC.4.1** (A conservative execution-height bound). *Let  $S$  be the bit-height bound in Theorem 3.3, with the root query included in the input length. For a fixed query, all rational numbers needed by a compact execution certificate, including its aggregate reward and dual value, have bit height  $O(NS + NL + \log(N + 1))$  under a common-denominator implementation. Exact forward/backward execution and auditing thus admit polynomial bit complexity on at most  $N(N + 1)$  reached pairs and  $M(N + 1)$  transitions.*

*Proof.* The discounted weight of a reached pair is a sum of probabilities of public paths that produce its price label. Each path contains at most  $N - 1$  edges. The same product-of-input-denominators argument as in Theorem 3.3 bounds its rational denominator by  $D_0^{N+4}$  and its numerator by a comparable height, since its weight is at most one. This representation does not multiply the denominators of individual histories.

There are at most  $N + 1$  distinct numerical prices. A local action is either a physical endpoint or  $(r_v - a_v s)/q_v$  for one such price  $s$ . A local reward has degree at most two in that price, with rational primitive coefficients. A common denominator for all weighted reward contributions divides a fixed power of  $D_0^{N+4}$  times the product of the squared denominators of the distinct prices. Its logarithm is  $O(NL + NS)$ . The same denominator argument applies to payments, cap-multiplier contributions, bound multipliers, and the squared local dual terms; a difference of two prices introduces denominators already in this product. The sum contains only polynomially many compact-pair terms, and their magnitudes add at most polynomial factors and the bounded-height price magnitudes. This gives the stated height. Forward and backward dynamic programming on the reached graph then uses polynomially many operations on polynomial-height integers.  $\square$

This is a conservative implementation bound, not an assertion that the experimental total rewards attain it. In the implementation, reward and payment contributions are first grouped by effective price before their final sum. This reduces repeated greatest-common-divisor work across unrelated price denominators without changing the exact result. The independent audit reconstructs conditional payments directly from the action/transition records and checks local normal-cone conditions; it does not trust the compiler’s claimed response identity.

For trusted internally generated experiments, the worker explicitly lifts Python’s default decimal integer-string length guard. This affects serialization only, not mathematical arithmetic or input validity. The general compiler does not silently reinterpret floating-point inputs or accept arbitrary external executable data. The worker’s total reward can have many more decimal digits than a stored slope, precisely the distinction addressed by the proposition.

## EC.5 Behavioral Quotients and Information Accounting

A backward signature at a reached pair consists of its rational action and the ordered tuple of successor-vertex and successor-class indices. Two signatures are equal precisely when the corresponding complete suffix plans are equal. Because the graph is acyclic, successor classes are already known when a vertex is processed. The stored class index may be reused at other public vertices. The update map has the public transition as input and writes the successor's local class before the next action is decoded.

This timing convention is important. A past branch cannot be made available to the next action decoder as an additional uncharged register merely by calling it the last observed edge. Similarly, a remaining promise, past service tier, or retained seed would enlarge the observation model if made separately available. The minimal-alphabet theorem fixes these interfaces. It compares controllers with the same read-only program and current public observation, not architectures with different free history channels.

### EC.5.1 A strict reduction beyond counting numerical prices

Take a root fixed at zero, two equally likely intermediate vertices, and a common terminal vertex fixed at tier  $1/4$ . Intermediate rewards are  $2x - x^2/2$ , their caps are  $1/2$  and  $1$ , payments and discounting are one, and the root promise is  $3/4$ . Both branch caps bind. The intermediate actions are  $1/4$  and  $3/4$ , so two different prices can reach the terminal vertex. Yet its terminal action is fixed and has no successors. Both prices therefore belong to the same terminal behavioral class, and the entire optimal protocol can use one writable symbol. The exact test records two raw states at that vertex but a minimum alphabet of one. Counting numerical prices alone would overstate necessary memory.

### EC.5.2 Randomization and restricted-memory values

The Jensen argument in Theorem 4.1 concerns a randomized controller attaining the *full* optimum. It forces every action at every positive-probability public history to equal the unique optimum almost surely. It does not prove that randomization leaves all lower-alphabet optima unchanged. The frontier in Theorem 4.2 is consequently stated for deterministic encodings. These two results have different conclusions and should not be merged into an unsupported randomized-frontier claim.

## EC.6 Heterogeneous Cell Costs and Dispersion

For quadratic terminal reward  $f(c) = rc - qc^2/2$  and intermediate cost  $h_h(y) = \gamma_h y^2/2$ , the cell cost at minimum  $b_i$  expands as

$$C(i, j) = \sum_{h=i}^j \pi_h \{rb_h - qb_h^2/2 - rb_i + qb_i^2/2 + \gamma_h b_h^2/2 - \gamma_h b_h b_i + \gamma_h b_i^2/2\}.$$

Prefix sums of  $\pi_h$ ,  $\pi_h b_h$ ,  $\pi_h b_h^2$ ,  $\pi_h \gamma_h$ ,  $\pi_h \gamma_h b_h$ , and  $\pi_h \gamma_h b_h^2$  therefore evaluate a cell in constant arithmetic time. With general convex cost functions, the segmentation recurrence still applies after cell costs are computed; the quadratic prefix-sum preprocessing bound is not assigned to an arbitrary function oracle.

The finite enumeration test does not assume contiguous cells. It enumerates all set partitions for branch counts two through seven, computes each cell at its smallest cap, and compares the best value at every cell count with the ordered dynamic program. Two cost families are used: heterogeneous quadratic costs with quadratic terminal reward, and heterogeneous cubic costs with a strictly increasing piecewise-concave terminal reward. Probabilities are proportional to branch indices rather than equal. In total, 2,308 arbitrary partitions are checked exactly.

For the dispersion test, probabilities are again unequal, the mean cap is  $1/2$ , and centered branch indices define  $d_j$ . Four rational positive dispersion levels are tested for each branch count. The resulting 84 strict loss comparisons complement, but do not establish, the analytic monotonicity proof. The limiting alphabet jump at zero dispersion follows from the optimal terminal actions, not from a numerical precision threshold.

## EC.7 Complete Endpoint-Normal Splitting

The effective interval is the intersection of four half-spaces on one coordinate: physical lower and upper bounds and corridor lower and upper bounds. At a lower endpoint, its outward lower normal is generated by any original lower inequality attaining the maximum lower bound; at an upper endpoint, the upper normal is generated by an inequality attaining the minimum upper bound. At a singleton, both normal directions are present, even if supplied by different original inequalities. Thus their nonnegative sum is the entire required interval normal cone.

For example, let the physical upper bound be  $1/2$  and let the corridor upper bound also equal  $1/2$ . A positive effective upper multiplier  $u^+$  can be split as  $\theta u^+$  to the corridor and  $(1 - \theta)u^+$  to the physical bound for any  $0 \leq \theta \leq 1$ . Both original constraints are active, so stationarity and complementarity persist. The resulting supporting cut need not be unique. At zero corridor width, both corridor endpoints coincide; the residual's sign determines a valid orientation without dividing by that width. A physical upper bound coinciding with a corridor lower bound similarly creates a singleton, and the corresponding two opposite normals span every residual sign.

The boundary suite tests physical-first, corridor-first, and equal splitting. It includes ordinary coupled-graph queries, upper and lower physical ties, fixed physical tiers, zero corridor widths, and intersections reduced to one point by different physical and corridor bounds. For each feasible anchor, nearby rational table/width queries are solved and the proposed upper support is evaluated exactly. Infeasible test queries are rejected rather than counted as supporting-cut successes. The recorded 150 anchors and 1,185 inequalities are over feasible queries only.

## EC.8 Comparison Algorithms and Reproducibility

### EC.8.1 Input families

The binary comparison family has  $T$  dates, one root, two vertices at every later date, and uniform binary transitions. Its size is  $2T - 1$  public vertices and  $2^T - 1$  unfolded occurrences. All payment coefficients and discount factors equal one. Reward coefficients and positive quadratic coefficients are drawn from the fixed rational grids specified in `experiments.py` using seeds 13 and 29. A constant tier  $2/5$  is feasible; its root payment is  $2T/5$ . Each cap equals its remaining constant-tier payment plus a strictly positive rational slack. Thus the generic optimizer and the grid method can construct a feasible starting policy without knowing the optimum.

The broadly capped family uses the preserved rational generator with discount  $19/20$ , heterogeneous positive payment coefficients, and nonuniform positive transitions. Horizon values 16, 32, and 64 at width four are reported. A related tighter-cap instance uses 128 dates, caps  $(8 + v \bmod 5)/10$ , and root payment  $1/4$ ; its zero lower policy proves the lower envelope feasible. These cases can have many distinct active cap barriers.

The late-renewal family has width four and discount  $19/20$ . Most caps equal the remaining maximum physical payment and are redundant. At the penultimate date, the four caps are  $1/2, 3/5, 7/10, 4/5$ , the local reward coefficient is zero, and the terminal reward coefficient is two. Earlier reward coefficients cycle through  $0, 1/2, 1$ . All quadratic and payment coefficients equal one. A constant tier  $1/4$  is feasible and determines the exact root promise. The heterogeneous penultimate caps bind in the reported optima and generate four distinct required terminal tiers after recombination. This is a deliberately sparse-activation scale family, not a surrogate for a worst-case cap pattern.

The dense-knot family has two vertices per later date, discount and payment coefficients one, quadratic coefficients one, slack caps equal to remaining horizon, and distinct rational local reward coefficients. The formula in the generator separates local clipping thresholds. Its response family exhibits quadratic storage growth while its execution has no active cap memory. Reporting both this family and the late-renewal family avoids conflating execution memory with response density.

### EC.8.2 Independent allocation and generic convex optimization

The exact tree baseline stores separate occurrence-weighted responses and subtree caps in the allocation units  $z_h = w_h a_v x_h$ . It forms its marginal response by evaluating at the union of local and child knots and interpolating affine segments. It never imports the quotient compiler, and it does not memoize public vertices. This is a transparent implementation of a tree marginal-reduction scheme. It is not certified as a line-by-line implementation of Mjelde’s or Tang’s published code, and no strongest-known general laminar running time is attributed to it.

The numerical baseline expands the occurrence decision vector, uses Hessian diagonal entries  $w_h q_v$  and linear objective entries  $-w_h r_v$ , and imposes normalized continuation-cap rows, the root equality, and physical bounds. It uses SciPy 1.17.0 (Virtanen et al., 2020), analytic gradients, an exact sparse diagonal Hessian, and sparse linear Jacobians. The reported protocol sets gradient, step, and barrier tolerances to

$10^{-12}$ , with initial barrier parameter and initial barrier tolerance  $10^{-10}$ , and a maximum of 1,000 iterations. Every returned status, iteration count, primal residual, stationarity residual, matrix size, and absolute value discrepancy is retained. The barrier initialization was tightened in a pilot before the final comparison; the pilot outputs remain available and are not substituted for the final table. OSQP was not available in the execution environment, so no OSQP run or commercial-solver comparison is claimed.

### EC.8.3 Public-graph promise discretization

At vertex  $v$ , the primal state is the remaining promise  $y$ . Successor promises are restricted to multiples of  $1/Q$ . With uniform binary successors, the expected future promise is their average; the current action is the exact residual  $x = y - (y_1 + y_2)/2$ . Values are computed backward at each public vertex. A max-plus convolution forms the best future reward for each possible sum of the two successor grid indices, reducing repeated successor-allocation work. The root query uses its stipulated rational promise, not a rounded grid approximation.

The objective comparisons in this baseline use double precision. After a maximizing grid index is chosen, the policy is replayed using exact rational arithmetic: conditional promises, every action bound and cap, the root payment, and the implemented reward are checked. Thus the reported grid policy is exactly feasible, but the numerical maximization is not advertised as a proof of exact optimality even within its discretization. Its loss is measured against the exact continuous optimum. Nested-grid feasibility allows a finer grid to reproduce a coarser policy, and the specific regression values are checked to be nondecreasing.

This is a state-space alternative on the public graph, not an SDDP implementation. The SDDP literature supplies more general iterative approximations and convergence results (Pereira and Pinto, 1991; Girardeau et al., 2015; Dowson, 2020). A single-query demand for an exact rational certificate is a different stopping/output specification from a sampled tolerance benchmark. No relative speed assertion about those unimplemented methods is made. The current experiment compares public-state discretization with finite exact dual closure under explicitly common inputs.

### EC.8.4 Timing and memory ledger

The numerical environment is Python 3.13.5, NumPy 2.3.5, and SciPy 1.17.0 on the recorded Linux worker. BLAS/OpenMP thread counts are set to one; workers are sequential, not concurrent. This is a shared provisioned environment, not a dedicated hardware benchmark. Timings exclude common numerical-library imports and use `perf_counter`. Raw runs, not only medians, are stored. An absolute process peak includes imports, input construction, response storage, and other retained objects, so it cannot by itself identify the incremental space cost of a small algorithm.

There are 54 completed comparison workers and 12 completed scale workers. Comparison phases are repeated three times. In the scale workers, the single-query certificate and audit are measured once; all table notes disclose that distinction. Whole-response verification on 35 inherited cases and separate small-instance tests establishes a different audit scope. A pilot whole-response audit of a larger graph and an exploratory tighter-cap 1,021-vertex worker exceeded their imposed overall run limits; these censored

attempts are recorded separately and are not assigned completed runtimes or successful certificates. They caution against treating a graph-sized operation bound as uniformly cheap exact arithmetic.

The 101-query timing in Table 1 reuses the root response and tests payments evenly spaced across its feasible interval. It excludes new policy generation and certificate audit. Neither the numerical inversion times nor the small writable alphabet remove the cost of storing rational read-only tables or recompiling after primitive changes.

### EC.8.5 Commands and output integrity

From a clean repository checkout with the declared Python dependencies, run the R30 `tests.py`, then `experiments.py -suite comparisons`, `experiments.py -suite scaling`, and `summarize.py`. The build script compiles the main manuscript and this companion and refreshes only their own cross-reference labels. Saved worker outputs permit resuming a completed specification; an explicit rerun mode removes those caches before measuring anew. The source and output manifest records hashes. A separate final validation regenerates certificates and checks the current source against saved values and curves; timing variation is not treated as a correctness failure.

## References

- Dowson O (2020) The policy graph decomposition of multistage stochastic programming problems. *Networks* 76(1):3–23. doi:10.1002/net.21932.
- Federgruen A, Groenevelt H (1986) The greedy procedure for resource allocation problems: Necessary and sufficient conditions for optimality. *Operations Research* 34(6):909–918. doi:10.1287/opre.34.6.909.
- Girardeau P, Leclère V, Philpott AB (2015) On the convergence of decomposition methods for multistage stochastic convex programs. *Mathematics of Operations Research* 40(1):130–145. Published online July 16, 2014. doi:10.1287/moor.2014.0664.
- Groenevelt H (1991) Two algorithms for maximizing a separable concave function over a polymatroid feasible region. *European Journal of Operational Research* 54(2):227–236. doi:10.1016/0377-2217(91)90300-K.
- Pereira MVF, Pinto LMVG (1991) Multi-stage stochastic optimization applied to energy planning. *Mathematical Programming* 52:359–375. doi:10.1007/BF01582895.
- Virtanen P, Gommers R, Oliphant TE, et al. (2020) SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods* 17:261–272. doi:10.1038/s41592-019-0686-2.

Table 1: Repeated-query and execution-machine measurements

| Vertices | 101 root inversions | Machine minimization |
|----------|---------------------|----------------------|
| 253      | 0.0014              | 0.0044               |
| 509      | 0.0022              | 0.0129               |
| 1021     | 0.0043              | 0.0509               |
| 2045     | 0.0087              | 0.2075               |

Seconds, median of three repetitions. The 101 payments are equally spaced across the compiled feasible root interval. Only inversion is included; emitting and auditing 101 different policies is not included. Machine minimization is for the original single query.